# CHAPTER 03. 시리즈로 1차원 배열 다루기

# 목차

1. 데이터 입력과 시리즈 객체의 정보 확인
2. 시리즈 객체의 슬라이싱
3. 시리즈 객체의 산술 연산
4. 시리즈 객체의 내용 변경

# 01 데이터 입력과 시리즈 객체의 정보 확인

## ▪ 개요

- 데이터 분석 작업을 요리에 비유하면, 판다스는 만능 조리도구와 같다.
- 조리도구를 능숙하게 다룰 수 있어야 맛있는 요리를 만들 수 있듯이, 판다스를 잘 활용해야 효과적인 데이터 분석이 가능하다.
- 판다스의 시리즈(Series)는 1차원 배열 형태의 데이터를 저장하고 처리하는 데 사용된다.

표 3-1 전국 월평균 기온

| 월 | 1월 | 2월 | 3월 | 4월 | 5월 | 6월 | 7월 | 8월 | 9월 | 10월 | 11월 | 12월 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 기온 | −0.8 | −0.1 | 7.7 | 13.8 | 18.0 | 22.4 | 25.9 | 25.3 | 21.0 | 14.0 | 9.6 | −1.4 |

# 01 데이터 입력과 시리즈 객체의 정보 확인

## ■ 데이터 입력

```
[코드 3-1]
import pandas as pd
import numpy as np

# 자료의 입력
temp = pd.Series([-0.8, -0.1, 7.7, 13.8, 18.0, 22.4,
                  25.9, 25.3, 21.0, 14.0, 9.6, -1.4])
temp                                                # temp의 내용 확인

# 월이름을 레이블 인덱스로 지정하기
temp.index                                          # 인덱스 내용 확인
temp.index = ['1월','2월','3월','4월',              # 월이름을 인덱스로 지정
              '5월','6월','7월','8월',
              '9월','10월','11월','12월']
temp                                                # temp의 내용 확인
```

# 01 데이터 입력과 시리즈 객체의 정보 확인

```
>>> temp = pd.Series([-0.8, -0.1, 7.7, 13.8, 18.0, 22.4,
... 25.9, 25.3, 21.0, 14.0, 9.6, -1.4])
>>>
>>> temp
0     -0.8
1     -0.1
2      7.7
3     13.8
4     18.0
5     22.4
6     25.9
7     25.3
8     21.0
9     14.0
10     9.6
11    -1.4
dtype: float64
```

- 12개월의 기온 자료 왼쪽에 절대 위치 인덱스 값(0~11)이 자동으로 레이블로 부여된 것을 확인할 수 있다.

# 01 데이터 입력과 시리즈 객체의 정보 확인

```
>>> # 월이름을 레이블 인덱스로 지정하기
>>>
>>> temp.index # 인덱스 내용 확인
RangeIndex(start=0, stop=12, step=1)
```

- 현재 지정된 인덱스는 temp.index를 출력하여 확인할 수 있다.
  - 0~11이 레이블 인덱스로 지정됨
  - 사용자가 레이블을 지정하지 않으면, 0부터 시작하는 정수가 자동으로 레이블 인덱스로 할당된다.

# 01 데이터 입력과 시리즈 객체의 정보 확인

```
>>> temp.index = ['1월','2월','3월','4월',        # 월이름을 인덱스로 지정
...               '5월','6월','7월','8월',
...               '9월','10월','11월','12월']
>>> temp
1월     -0.8
2월     -0.1
3월      7.7
4월     13.8
5월     18.0
6월     22.4
7월     25.9
8월     25.3
9월     21.0
10월    14.0
11월     9.6
12월    -1.4
dtype: float64
```

- 0~11의 정수 대신 월 이름이 레이블 인덱스로 출력된다.
  - 절대 위치 인덱스는 내부적으로 관리되므로, 레이블 인덱스를 지정하더라도 유지된다.

# 01 데이터 입력과 시리즈 객체의 정보 확인

## ■ 시리즈 객체의 정보 확인

```
[코드 3-2]
# 코드 3-1에 이어서 실행

# 시리즈 객처의 정보 확인
temp.size                                   # 배열의 크기(값의 개수)
len(temp)                                   # 배열의 크기(값의 개수)
temp.shape                                  # 배열의 형태
```

# 01 데이터 입력과 시리즈 객체의 정보 확인

```
>>> temp.size                                    # 배열의 크기(값의 개수)
12
>>> len(temp)                                    # 배열의 크기(값의 개수)
12
```

- temp의 크기 확인:
  - temp.size 속성을 사용하여 값의 개수를 확인한다.
    - » temp.size()가 아니라 temp.size로 접근해야 한다.
- 파이썬 내장 함수 len()을 사용하여 크기를 확인할 수도 있다.

```
>>> temp.shape                                   # 배열의 형태
(12,)
```

- temp의 형태(shape) 확인:
  - temp.shape 속성을 사용하여 배열의 차원을 확인한다.
    - » (4,3): 4행 3열의 2차원 배열을 의미
    - » (12,): 값이 12개 포함된 1차원 배열을 의미하며, 시리즈(Series)는 1차원 구조이므로 열(column) 정보는 생략된다.

# 01 데이터 입력과 시리즈 객체의 정보 확인

```
>>> temp.dtype                                    # 배열의 자료형
dtype('float64')
```

- dtype 속성은 배열의 자료형을 저장한다.
  - 'float64'는 소수점을 포함하는 실수형 숫자를 의미한다.

# 02 시리즈 객체의 슬라이싱

## ■ 인덱싱과 슬라이싱

- **인덱싱(Indexing):** 시리즈에 저장된 값의 위치를 가리키는 체계를 의미한다.
- **슬라이싱(Slicing):** 인덱스와 조건문을 이용하여 시리즈에서 일부 값을 잘라내어 추출하는 연산이다.
- 시리즈 학습에서 가장 핵심적인 개념 중 하나이다.

| temp | -0.8 | -0.1 | 7.7 | 13.8 | 18.0 | 22.4 | 25.9 | 25.3 | 21.0 | 14.0 | 9.6 | -1.4 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 절대 위치 인덱스 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 레이블 인덱스 | 1월 | 2월 | 3월 | 4월 | 5월 | 6월 | 7월 | 8월 | 9월 | 10월 | 11월 | 12월 |

그림 3-1 temp 객체의 절대 위치 인덱스와 레이블 인덱스

# 02 시리즈 객체의 슬라이싱

[코드 3-3]

```
# 코드 3-1에 이어서 실행

# 인덱싱과 슬라이싱
temp.iloc[2]                                # 인덱스 2의 값
temp.loc['3월']                             # 인덱스 '3월'의 값
temp.iloc[[3,5,7]]                          # 인덱스 3,5,7의 값
temp.loc[['4월','6월','8월']]               # 인덱스 '4월','6월','8월'의 값

temp.iloc[5:8]                              # 인덱스 5~7의 값
temp.loc['6월':'9월']                       # 인덱스 '6월'~'9월'의 값
temp.iloc[:4]                               # 인덱스 0~3의 값
temp.iloc[9:]                               # 인덱스 9~11의 값
temp.iloc[:]                                # 인덱스 0~11의 값
```

# 02 시리즈 객체의 슬라이싱

```
>>> temp.iloc[2]                                    # 인덱스 2의 값
np.float64(7.7)
>>> temp.loc['3월']                                 # 인덱스 '3월'의 값
np.float64(7.7)
```

- 판다스에서는 절대 위치 인덱스와 레이블 인덱스를 이용하여 값의 위치를 지정한다.
  - 절대 위치 인덱스 사용: 시리즈객체명.iloc[pos]
  - 레이블 인덱스 사용: 시리즈객체명.loc[pos]
- 판다스는 레이블 인덱스를 중요시하므로 temp.loc['4월']은 temp['4월']과 같은 간단한 형태로 사용 가능하다.

# 02 시리즈 객체의 슬라이싱

```
>>> temp.iloc[[3,5,7]]                          # 인덱스 3,5,7의 값
4월    13.8
6월    22.4
8월    25.3
dtype: float64
>>> temp.loc[['4월','6월','8월']]                # 인덱스 '4월','6월','8월'의 값
4월    13.8
6월    22.4
8월    25.3
dtype: float64
```

- 인덱스를 여러 개 동시에 지정할 경우, 인덱스 값들을 리스트([ ]로 묶어서 입력한다.

# 02 시리즈 객체의 슬라이싱

```
>>> temp.iloc[5:8]                                  # 인덱스 5~7의 값
6월    22.4
7월    25.9
8월    25.3
dtype: float64
>>> temp.loc['6월':'9월']                           # 인덱스 '6월'~'9월'의 값
6월    22.4
7월    25.9
8월    25.3
9월    21.0
dtype: float64
```

- 연속된 인덱스를 지정할 경우, 범위 연산자(:)를 사용할 수 있다.
- 형태: start:end
  - 절대 위치 인덱스(.iloc[]): start는 포함되지만, end는 포함되지 않는다.
  - 레이블 인덱스(.loc[]): start와 end 모두 포함된다.

# 02 시리즈 객체의 슬라이싱

```
>>> temp.iloc[:4]                                    # 인덱스 0~3의 값
1월    -0.8
2월    -0.1
3월     7.7
4월    13.8
dtype: float64
>>> temp.iloc[9:]                                    # 인덱스 9~11의 값
10월    14.0
11월     9.6
12월    -1.4
dtype: float64
>>> temp.iloc[:]                                     # 인덱스 0~11의 값
1월     -0.8
2월     -0.1
(생략)
11월     9.6
12월    -1.4
dtype: float64
```

# 02 시리즈 객체의 슬라이싱

표 3-2 절대 위치 인덱스와 레이블 인덱스에서 경곗값의 포함 여부

| 인덱스 | start | end | 예 |
|---|---|---|---|
| 절대 위치 인덱스 | ○ | × | iloc[5:8] → 5, 6, 7 |
| 레이블 인덱스 | ○ | ○ | loc['6월':'9월'] → '6월', '7월', '8월', '9월' |

- 범위 지정 연산(start:end)에서 start나 end를 생략할 수 있다.

– start 생략: end 이전의 모든 인덱스를 포함

– end 생략: start부터 이후의 모든 인덱스를 포함

– start와 end 모두 생략: 전체 인덱스를 포함

# 02 시리즈 객체의 슬라이싱

## ■ 조건문을 이용한 슬라이싱

- 조건문을 사용하여 특정 조건에 맞는 값들을 시리즈 객체에서 추출할 수 있다.
- 조건문을 이용한 슬라이싱 시 .loc[] 메서드를 사용해야 한다.

| 조건문 | 의미 |
|---|---|
| a > b | a가 b보다 큰가? |
| a >= b | a가 b보다 크거나 같은가? |
| a < b | a가 b보다 작은가? |
| a <= b | a가 b보다 작거나 같은가? |
| a == b | a와 b가 같은가? |
| a != b | a와 b가 같지 않은가? |

# 02 시리즈 객체의 슬라이싱

**[코드 3-4]**

```
# 코드 3-1에 이어서 실행


# 월평균 기온이 15도 이상인 월들의 기온을 보이시오
temp.loc[temp >= 15]
# 월평균 기온이 15도 이상 25도 미만인 월들의 기온을 보이시오
temp.loc[(temp >= 15) & (temp < 25)]
# 월평균 기온이 5도 미만이거나 25도 이상인 월들의 기온을 보이시오
temp.loc[(temp < 5) ¦ (temp >= 25)]
# 3월보다 기온이 낮은 월들의 기온을 보이시오
march = temp.loc['3월']
temp.loc[temp < march]
# where() 를 이용한 조건 검색
temp.where(temp >= 15)
temp.where(temp >= 15).dropna()
```

# 02 시리즈 객체의 슬라이싱

```
>>> # 월평균 기온이 15도 이상인 월들의 기온을 보이시오
>>>
>>> temp.loc[temp >= 15]
5월    18.0
6월    22.4
7월    25.9
8월    25.3
9월    21.0
dtype: float64
```

- 조건문의 비교 결과가 True인 값들만 추출된다.

# 02 시리즈 객체의 슬라이싱

| temp | | temp >= 15 | | temp[temp >= 15] | |
|---|---|---|---|---|---|
| 1월 | -0.8 | 1월 | False | | |
| 2월 | -0.1 | 2월 | False | | |
| 3월 | 7.7 | 3월 | False | | |
| 4월 | 13.8 | 4월 | False | | |
| 5월 | 18.0 | 5월 | True | 5월 | 18.0 |
| 6월 | 22.4 | 6월 | True | 6월 | 22.4 |
| 7월 | 25.9 | 7월 | True | 7월 | 25.9 |
| 8월 | 25.3 | 8월 | True | 8월 | 25.3 |
| 9월 | 21.0 | 9월 | True | 9월 | 21.0 |
| 10월 | 14.0 | 10월 | False | | |
| 11월 | 9.6 | 11월 | False | | |
| 12월 | -1.4 | 12월 | False | | |

그림 3-2 조건을 이용한 슬라이싱의 실행 과정

# 02 시리즈 객체의 슬라이싱

```
>>> # 월평균 기온이 15도 이상 25도 미만인 월들의 기온을 보이시오
>>>
>>> temp.loc[(temp >= 15) & (temp < 25)]
5월    18.0
6월    22.4
9월    21.0
dtype: float64
>>> # 월평균 기온이 5도 미만이거나 25도 이상인 월들의 기온을 보이시오
>>>
>>> temp.loc[(temp < 5) | (temp >= 25)]
1월     -0.8
2월     -0.1
7월     25.9
8월     25.3
12월    -1.4
dtype: float64
```

# 02 시리즈 객체의 슬라이싱

- 값을 추출하기 위한 조건 지정 시 하나의 조건이 아닌 여러 개의 조건을 지정할 수 있다.
- 각 조건은 '&' (and)나 ':' (or)로 연결한다.
- 주의할 점은 각 조건은 반드시 ( ) 로 묶어 주어야 한다는 것
- ( )가 없는 다중 조건문 temp >= 15 & temp < 25 은 에러 발생

```
temp >= 15 & temp < 25
```

```
(temp >= 15) & (temp < 25)
```

# 02 시리즈 객체의 슬라이싱

```
>>> # 3월보다 기온이 낮은 월들의 기온을 보이시오
>>>
>>> march = temp.loc['3월']
>>> temp.loc[temp < march]
1월    -0.8
2월    -0.1
12월   -1.4
dtype: float64
```

- 조건문에서 비교 대상 값을 지정할 때 배열 객체에 저장된 특정한 값도 가져다 사용할 수 있다. 조건문 temp < march는 temp < 7.7과 동일한 의미

# 02 시리즈 객체의 슬라이싱

```
>>> # where() 를 이용한 조건 검색
>>>
>>> temp.where(temp >= 15)
1월      NaN
2월      NaN
3월      NaN
4월      NaN
5월     18.0
6월     22.4
7월     25.9
8월     25.3
9월     21.0
10월     NaN
11월     NaN
12월     NaN
dtype: float64
>>> temp.where(temp >= 15).dropna()
5월    18.0
6월    22.4
7월    25.9
8월    25.3
9월    21.0
dtype: float64
```

- where() 메서드는 조건문을 이용하여 값을 검색하는 기능을 제공한다.
- 조건에 맞지 않는 값들은 결측값(NaN)으로 표시된다.
- 결측값을 제외하고 조건에 맞는 값들만 보려면 dropna() 메서드를 where() 실행 결과에 추가한다.

# 02 시리즈 객체의 슬라이싱

## ■ 시리즈 객체에서 값의 위치 찾기

- 값이 아닌, 해당 값의 인덱스를 추출해야 하는 경우가 있다.
- 이때, 특정 값을 기준으로 인덱스를 찾는 방법을 활용할 수 있다.

**[코드 3-5]**

```
# 코드 3-1에 이어서 실행


# 월평균 기온이 15도 이상인 월들의 이름을 보이시오
temp.loc[temp >= 15]                    # 조건에 맞는 값 추출
temp.loc[temp >= 15].index              # 조건에 맞는 값들의 인덱스 추출
```

# 02 시리즈 객체의 슬라이싱

```
>>> # 월평균 기온이 15도 이상인 월들의 이름을 보이시오
>>>
>>> temp.loc[temp >= 15]                          # 조건에 맞는 값 추출
5월    18.0
6월    22.4
7월    25.9
8월    25.3
9월    21.0
dtype: float64
>>> temp.loc[temp >= 15].index                    # 조건에 맞는 값들의 인덱스 추출
Index(['5월', '6월', '7월', '8월', '9월'], dtype='object')
```

# 03 시리즈 객체의 산술 연산

## ■ 시리즈 객체에 대한 산술 연산

- 시리즈 객체를 대상으로 산술 연산을 수행할 수 있다.
- 시리즈 객체의 산술 연산은 각각의 값에 대해 개별적으로 적용된다.
- 두 시리즈 객체 간의 산술 연산은 인덱스가 같은 값들끼리 수행된다.

[코드 3-6]

```
# 코드 3-1에 이어서 실행



temp + 1
2 * temp + 0.1
temp + temp
temp.loc[temp >= 15] + 1      # 기온이 15도 넘는 월들에 대해서만 1 증가
```

# 03 시리즈 객체의 산술 연산

```
>>> >>> # 시리즈 객체에 대한 산술 연산
>>>
>>> temp + 1
1월      0.2
2월      0.9
(생략)
11월    10.6
12월    -0.4
dtype: float64
>>> 2 * temp + 0.1
1월     -1.5
2월     -0.1
(생략)
11월    19.3
12월    -2.7
dtype: float64
```

**[참고]**
temp의 내용이
변경되지는 않음

# 03 시리즈 객체의 산술 연산

```
>>> temp + temp
1월     -1.6
2월     -0.2
(생략)
11월    19.2
12월    -2.8
dtype: float64
>>> temp.loc[temp >= 15] + 1     # 기온이 15도 넘는 월들에 대해서만 1 증가
5월    19.0
6월    23.4
7월    26.9
8월    26.3
9월    22.0
dtype: float64
```

**[참고]**
temp의 내용이
변경되지는 않음

- temp + 1의 실행 결과를 temp에 실제로 반영하려면 temp = temp + 1 또는 temp += 1 형태로 실행해야 한다.

# 03 시리즈 객체의 산술 연산

표 3-3 시리즈 객체 temp에 대한 산술 연산

| temp | | temp + 1 | | temp + temp | |
|---|---|---|---|---|---|
| 1월 | -0.8 | 1월 | 0.2 | 1월 | -1.6 |
| 2월 | -0.1 | 2월 | 0.9 | 2월 | -0.2 |
| 3월 | 7.7 | 3월 | 8.7 | 3월 | 15.4 |
| 4월 | 13.8 | 4월 | 14.8 | 4월 | 27.6 |
| 5월 | 18.0 | 5월 | 19.0 | 5월 | 36.0 |
| 6월 | 22.4 | 6월 | 23.4 | 6월 | 44.8 |
| 7월 | 25.9 | 7월 | 26.9 | 7월 | 51.8 |
| 8월 | 25.3 | 8월 | 26.3 | 8월 | 50.6 |
| 9월 | 21.0 | 9월 | 22.0 | 9월 | 42.0 |
| 10월 | 14.0 | 10월 | 15.0 | 10월 | 28.0 |
| 11월 | 9.6 | 11월 | 10.6 | 11월 | 19.2 |
| 12월 | -1.4 | 12월 | -0.4 | 12월 | -2.8 |

# 03 시리즈 객체의 산술 연산

## ■ 시리즈의 통계 관련 메서드

- 시리즈 객체는 데이터 분석에 사용되는 다양한 통계 관련 메서드들을 포함한다.

표 3-4 시리즈의 통계 관련 메서드

| 메서드명 | 설명 |
|---|---|
| sum() | 합계 |
| mean() | 평균 |
| median() | 중앙값 |
| max() | 최댓값 |
| min() | 최솟값 |
| std() | 표준편차 |
| var() | 분산 |

# 03 시리즈 객체의 산술 연산

**[코드 3-7]**

```
# 코드 3-1에 이어서 실행


temp.sum()                                  # 값들의 합계
temp.mean()                                 # 값들의 평균
temp.median()                               # 값들의 중앙값
temp.max()                                  # 값들의 최대값
temp.min()                                  # 값들의 최소값
temp.std()                                  # 값들의 표준편차
temp.var()                                  # 값들의 분산
temp.abs()                                  # 값들의 절대값
temp.describe()                             # 기초통계정보
```

# 03 시리즈 객체의 산술 연산

```
>>> temp.sum()                                    # 값들의 합계
np.float64(155.39999999999998)
>>> temp.mean()                                   # 값들의 평균
np.float64(12.949999999999998)
>>> temp.median()                                 # 값들의 중앙값
np.float64(13.9)
>>> temp.max()                                    # 값들의 최대값
np.float64(25.9)
>>> temp.min()                                    # 값들의 최소값
np.float64(-1.4)
>>> temp.std()                                    # 값들의 표준편차
np.float64(10.026011624315478)
>>> temp.var()                                    # 값들의 분산
np.float64(100.52090909090909)
```

# 03 시리즈 객체의 산술 연산

```
>>> temp.abs()                                    # 값들의 절대값
1월      0.8
2월      0.1
(생략)
11월     9.6
12월     1.4
dtype: float64
>>> temp.describe()                               # 기초통계정보
count    12.000000
mean     12.950000
std      10.026012
min      -1.400000
25%       5.750000
50%      13.900000
75%      21.350000
max      25.900000
dtype: float64
```

# 03 시리즈 객체의 산술 연산

표 3-5 describe()에 포함된 통계값

| 통계값 | 설명 |
|---|---|
| count | 값들의 개수 |
| mean | 값들의 평균 |
| std | 값들의 표준편차 |
| min | 값들의 최솟값 |
| 25% | 1사분위수 |
| 50% | 2사분위수(중앙값) |
| 75% | 3사분위수 |
| max | 값들의 최댓값 |

# 04 시리즈 객체의 내용 변경

## ■ 시리즈 객체에 대한 값의 수정·추가·삭제

```
[코드 3-8]
import pandas as pd

salary = pd.Series([20, 15, 18, 30])        # 레이블 인덱스가 없는 시리즈 객체
score = pd.Series([75, 80, 90, 60],
                  index=['KOR', 'ENG', 'MATH', 'SOC'])
salary
score

# 값의 변경
score.iloc[0] = 85                          # 인덱스 0 의 값을 변경
score
score.loc['SOC'] = 65                       # 인덱스 'SOC' 의 값을 변경
score
score.loc[['ENG','MATH']] = [70,80]         # 인덱스 'ENG','MATH' 의 값을 변경
Score
```

# 04 시리즈 객체의 내용 변경

```
>>> salary
0    20
1    15
2    18
3    30
dtype: int64

>>> score
KOR     75
ENG     80
MATH    90
SOC     60
dtype: int64
```

# 04 시리즈 객체의 내용 변경

```
>>> score.iloc[0] = 85                          # 인덱스 0 의 값을 변경
>>> score
KOR     85
ENG     80
MATH    90
SOC     60
dtype: int64
>>> score.loc['SOC'] = 65                       # 인덱스 'SOC' 의 값을 변경
>>> score
KOR     85
ENG     80
MATH    90
SOC     65
dtype: int64
```

```
>>> score
KOR     75
ENG     80
MATH    90
SOC     60
```

# 04 시리즈 객체의 내용 변경

```
>>> score.loc[['ENG','MATH']] = [70,80]        # 인덱스 'ENG','MATH' 의 값을 변경
>>> score
KOR     85
ENG     70
MATH    80
SOC     65
dtype: int64
```

```
>>> score
KOR     85
ENG     80
MATH    90
SOC     65
```

- 값을 변경할 때 숫자 인덱스와 레이블 인덱스를 모두 사용할 수 있다.
- 여러 개의 값을 동시에 변경하려면:
  - 변경할 값들의 인덱스를 리스트로 묶어 입력
  - 변경할 값들도 리스트로 묶어 할당

# 04 시리즈 객체의 내용 변경

```
[코드 3-9]
# 코드3-8에 이어서 실행
# 값의 추가 (레이블 인덱스가 있는 경우)
Score.loc[ ' PHY ' ] = 50                    # 없는 인덱스 추가
Score
Score.iloc[5] = 90                        # 에러 발생
# 값의 추가 (레이블 인덱스가 없는 경우)
Next_idx = salary.size
Salary.iloc[next_idx] = 33                # 에러 발생
Salary.loc[next_idx] = 33                 # 정상 수행
Salary
```

# 04 시리즈 객체의 내용 변경

```
# _append() 메서드를 이용한 값의 추가
New = pd.Series({ ' MUS ' : 95})
Score._append(new)                          # score 변경 없음
Score
Score = score._append(new)                  # score 변경됨
Score

Salary._append(pd.Series([66]), ignore_index=True)
Salary = salary._append(pd.Series([66]), ignore_index=True)
Salary
```

# 04 시리즈 객체의 내용 변경

```
>>> # 값의 추가 (레이블 인덱스가 있는 경우)
>>>
>>> score.loc['PHY'] = 50                          # 없는 인덱스 추가
>>> score
KOR     85
ENG     70
MATH    80
SOC     65
PHY     50
dtype: int64
>>> score.iloc[5] = 90                             # 에러 발생
(생략)
IndexError: iloc cannot enlarge its target object
```

- .iloc[]에 숫자 인덱스를 지정하여 새로운 값을 추가하려면 에러가 발생.
- .iloc[]를 통해서는 배열의 크기를 늘릴 수 없음

# 04 시리즈 객체의 내용 변경

```
>>> # 값의 추가 (레이블 인덱스가 없는 경우)
>>>
>>> next_idx = salary.size
>>> salary.iloc[next_idx] = 33                    # 에러 발생
(생략)
IndexError: iloc cannot enlarge its target object
>>> salary.loc[next_idx] = 33                     # 정상 수행
>>> salary
0    20
1    15
2    18
3    30
4    33
dtype: int64
```

- 값의 추가는 loc[] (레이블 인덱스)를 통해서만 가능하다.

# 04 시리즈 객체의 내용 변경

```
>>> # _append() 메서드를 이용한 값의 추가
>>>
>>> new = pd.Series({'MUS': 95})
>>> score._append(new)                          # score 변경 없음
KOR     85
ENG     70
MATH    80
SOC     65
PHY     50
MUS     95
dtype: int64
>>> score
KOR     85
ENG     70
MATH    80
SOC     65
PHY     50
dtype: int64
```

_append()는 score 의 내용을 변경하지 않는다.

# 04 시리즈 객체의 내용 변경

```
>>> score = score._append(new)                    # score 변경됨
>>> score
KOR     85
ENG     70
MATH    80
SOC     65
PHY     50
MUS     95
dtype: int64
```

- 시리즈 객체의 마지막 부분에 새로운 값을 추가하려면 _append() 메서드를 사용한다.
- 단, _append() 메서드의 입력값은 판다스 시리즈 객체여야 한다.

# 04 시리즈 객체의 내용 변경

```
>>> salary._append(pd.Series([66]), ignore_index=True)
0    20
1    15
2    18
3    30
4    33
5    66
dtype: int64
>>> salary = salary._append(pd.Series([66]), ignore_index=True)
>>> salary
0    20
1    15
2    18
3    30
4    33
5    66
dtype: int64
```

레이블을 지정하지 않은
시리즈 객체의 경우
새로운 값을 추가하는 예

# 04 시리즈 객체의 내용 변경

- 레이블을 지정하지 않은 시리즈 객체에 새로운 값을 추가할 때는 ignore_index=True를 반드시 설정해야 한다.
- 이 설정을 적용하면:
- 기존 시리즈 객체의 레이블 인덱스를 무시하고, 추가되는 객체에 맞춰 새로운 레이블 인덱스를 부여한다.

# 04 시리즈 객체의 내용 변경

**[코드 3-10]**

```
# 코드3-8에 이어서 실행

# 값의 삭제
salary = pd.Series([20, 15, 18, 30])  # 레이블 인덱스가 없는 시리즈 객체
score = pd.Series([75, 80, 90, 60],
                  index=['KOR', 'ENG', 'MATH', 'SOC'])
score.drop('PHY')                          # 레이블 인덱스가 있는 경우
score                                      # score의 내용 변동 없음
score = score.drop('PHY’)
score                                      # score의 내용 변경

salary = salary.drop(1)                    # 레이블 인덱스가 없는 경우
Salary
```

# 04 시리즈 객체의 내용 변경

```
>>> score = score.drop('PHY')
>>> score
KOR     85
ENG     70
MATH    80
SOC     65
MUS     95
dtype: int64
```

- drop() 메서드 자체는 score 의 내용을 변경시키지 않으므로 이와 같이 score.drop('PHY')의 결과를 score 에 저장해야 한다.

# 04 시리즈 객체의 내용 변경

```
>>> salary = salary.drop(1)                    # 레이블 인덱스가 숫자인 경우
>>> salary
0    20
2    18
3    30
4    33
5    66
dtype: int64
```

- 레이블 인덱스가 숫자인 경우는 위와 같이 따옴표 없이 인덱스 값을 입력한다.

- [참고]

| | | |
|---|---|---|
| `>>> score = score.drop('PHY')` | = | `>>> score.drop('PHY', inplace=True)` |
| `>>> salary = salary.drop(1)` | = | `>>> salary.drop(1, inplace=True)` |

# 04 시리즈 객체의 내용 변경

## ■ 시리즈 객체의 복사

- 시리즈 객체를 다른 변수에 복사하여 사용할 때는 주의가 필요하다.
- 대입(=) 연산자를 사용하면 동일한 객체를 참조하게 되어 원본이 변경될 수 있다.
- 따라서, 반드시 .copy() 메서드를 사용하여 복사해야 한다.

# 04 시리즈 객체의 내용 변경

```
[코드 3-11]
# 시리즈 객체의 복사
import pandas as pd

score_1 = pd.Series([80, 75, 90],
                    index=['KOR','ENG','MATH'])

score_2 = score_1            # score_1 과 score_2 는 동일한 객체
score_2.loc['KOR'] = 95
score_2
score_1

score_3 = score_1.copy()     # score_1 과 score_3 은 독립된 객체
score_3.loc['KOR'] = 70
score_3
score_1
```

# 04 시리즈 객체의 내용 변경

```
>>> score_2 = score_1            # score_1 과 score_2 는 동일한 객체
>>> score_2.loc['KOR'] = 95
>>> score_2
KOR     95
ENG     75
MATH    90
dtype: int64
>>> score_1
KOR     95
ENG     75
MATH    90
dtype: int64
```

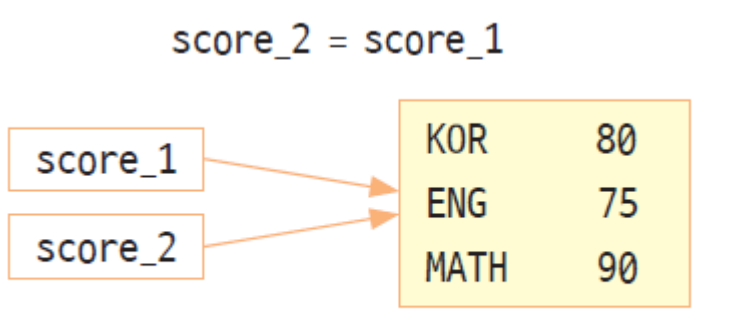


그림 3-3 scroe_1 객체의 복사: 대입 연산자 사용

- score_1과 score_2는 동일한 객체를 가리키므로, score_2를 변경하면 score_1도 함께 변경된다.
- 이러한 문제를 방지하려면 .copy() 메서드를 사용하여 복사해야 한다.

# 04 시리즈 객체의 내용 변경

```
>>> score_3 = score_1.copy()      # score_1 과 score_3 은 독립된 객체
>>> score_3.loc['KOR'] = 70
>>> score_3
KOR     70
ENG     75
MATH    90
dtype: int64
>>> score_1
KOR     95
ENG     75
MATH    90
dtype: int64
```

- 시리즈 객체를 복사할 때는 copy() 메서드를 사용한다.

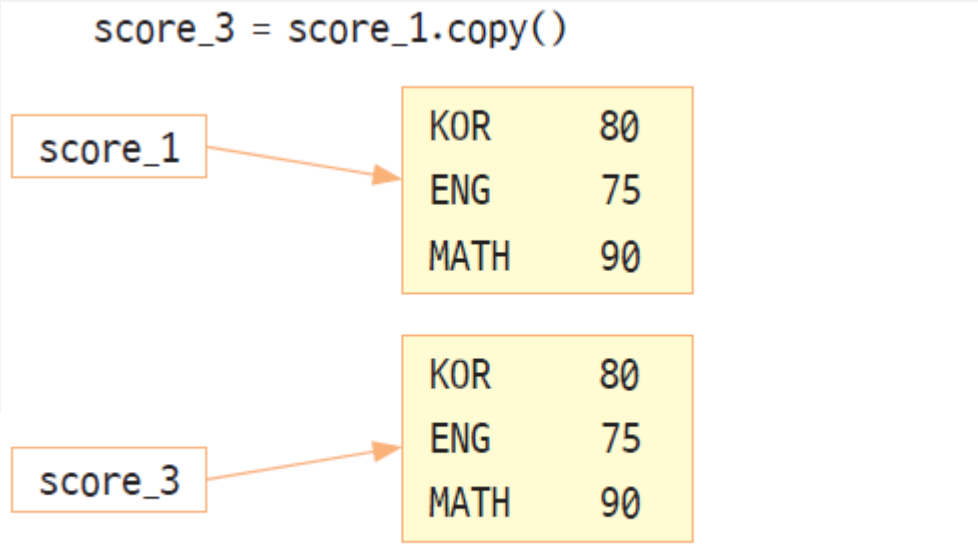


그림 3-4 scroe_1 객체의 복사: copy() 메서드를 사용한 객체 복사

# ♪♪ 실습해보기

- **※ 다음은 어느 보험회사 8개 영업팀(A~H)의 1분기 매출 실적 데이터이다. 각 문제를 해결하기 위한 파이썬 코드를 작성하시오.**

| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|
| 781 | 650 | 705 | 406 | 580 | 450 | 550 | 640 |

1. 매출 실적 데이터를 판다스 시리즈 sales에 저장하고, 팀 이름을 인덱스로 설정한 후 sales의 내용을 출력하시오.
2. sales의 배열 길이(값의 개수)를 출력하시오.
3. 인덱스 기준 두 번째 팀의 매출액을 출력하시오.
4. F팀의 매출액을 출력하시오.
5. 인덱스 기준 세 번째부터 다섯 번째 팀까지의 매출액을 출력하시오.
6. A, B, D팀의 매출액을 출력하시오.
7. 매출액이 500 미만이거나 700을 초과하는 팀들의 이름과 매출액을 출력하시오.

8. B팀보다 매출액이 많은 팀들의 이름과 매출액을 출력하시오.
9. 매출액이 600 미만인 팀들의 이름을 출력하시오.
10. 매출액이 600 미만인 팀들의 매출액을 20% 증가시켜 출력하시오.
11. 8개 팀의 매출액 평균, 합계, 표준편차를 출력하시오.
12. A, C팀의 매출액을 각각 810, 820으로 변경한 후 변경 여부를 확인하시오.
13. 신생팀 J팀의 매출액 400을 sales에 추가하시오.
14. J팀의 매출액 정보를 삭제하시오.
15. sales의 내용을 sales2에 복사한 후, sales2의 매출액을 각각 500씩 증가시키고, sales와 sales2의 내용을 비교하시오.